

Ottimizzazione Vincolata per il Curve Fitting

Analisi Numerica e Comparativa per la Previsione del Degrado delle Batterie

Claudio Nuncibello
Corso di Numerical Methods for Scientific Computing

30 giugno 2026

Sommario

Questo studio si concentra sull'applicazione di diversi metodi di ottimizzazione numerica (Gradient Descent, L-BFGS-B, Adam e Lion) per la risoluzione di un problema di curve fitting parametrico non lineare. Utilizzando i log sperimentali estratti dal dataset della NASA riguardante il degrado delle batterie agli ioni di litio, si intende dimostrare l'efficacia e la robustezza degli algoritmi Quasi-Newton su spazi di ottimizzazione convessi a bassa dimensionalità. Il lavoro contrappone i risultati ottenuti da metodologie classiche e fisicamente vincolate rispetto a quelli derivanti da approcci puramente black-box basati sul Deep Learning.

Indice

1	Introduzione e Analisi del Problema	2
1.1	Contesto Operativo	2
1.2	Origine e Caratteristiche del Dataset	2
1.3	Il Problema della Regressione Non Lineare	3
1.4	Obiettivo dello Studio	3
2	Modello Fisico e Vincoli	3
2.1	L'Equazione Esponenziale	3
2.2	Spazio di Ottimizzazione e Vincoli	4
3	Rassegna degli Algoritmi di Ottimizzazione Continua	4
3.1	Metodi del Primo Ordine: Gradient Descent (GD)	4
3.2	Metodi Adattivi e del Momento: Adam	5
3.3	Metodi Quasi-Newton: L-BFGS-B	5
3.4	Innovazioni Estreme: Lion	5
4	Architettura Software e Metodologia	5
4.1	Struttura del Codice	5
4.2	Pipeline Sperimentale	6
5	Analisi dei Risultati	6
5.1	Confronto delle Metriche Computazionali	6
5.2	Dinamica di Convergenza Geometrica e Traiettorie	7
6	Modello Fisico vs Deep Learning	8
6.1	Implementazione della Baseline Neurale (MLP)	8
6.2	Confronto Teorico-Pratico e Sovra-parametrizzazione	8

1 Introduzione e Analisi del Problema

1.1 Contesto Operativo

La previsione del ciclo di vita e dello stato di salute (*State of Health*, SOH) delle batterie agli ioni di litio rappresenta una delle sfide centrali nell'ingegneria dell'affidabilità moderna. Sistemi aerospaziali, veicoli elettrici e dispositivi elettronici dipendono criticamente dalla capacità di un accumulatore di mantenere la propria carica nel tempo. Nel presente studio viene impiegato il dataset pubblico *NASA Battery Cycle-Level Dataset*, che fornisce registrazioni sperimentali relative al degrado della capacità di diverse celle sottoposte a continui cicli di carica e scarica. La presenza di rumore fisico nelle misurazioni (come i fisiologici recuperi temporanei di capacità) rende questo dataset un banco di prova ideale per valutare la robustezza di vari algoritmi di ottimizzazione.

1.2 Origine e Caratteristiche del Dataset

Il dataset utilizzato in questo studio proviene dal *Prognostics Center of Excellence (PCoE)* della NASA. Nello specifico, si tratta del celebre dataset relativo all'invecchiamento delle batterie agli ioni di litio (Li-ion), ampiamente utilizzato in letteratura come benchmark per algoritmi di prognostica e stima del ciclo di vita.

Il file analizzato, `battery_cycle_level_dataset_CLEAN_FINAL.csv`, raccoglie i dati aggregati a livello di singolo ciclo di carica/scarica per un totale di 19 batterie distinte (tra cui le serie B0005, B0006, B0007, B0018, ecc.). Per ciascun ciclo di funzionamento di una batteria, il dataset traccia le seguenti caratteristiche:

- **battery_id**: Identificativo univoco della cella di batteria sotto test.
- **cycle**: L'indice progressivo del ciclo di carica/scarica (compreso tra 1 e un massimo globale di 168 cicli).
- **capacity**: La capacità di scarica della batteria, misurata in Ampere-ora (Ah). Questa rappresenta la variabile target principale y per il nostro problema di curve fitting, il cui andamento decrescente nel tempo testimonia il degrado progressivo dell'accumulatore.
- **voltage**: Il valore medio di tensione misurato durante il ciclo.
- **temperature**: La temperatura di esercizio registrata sulla cella in gradi Celsius ($^{\circ}\text{C}$).
- **soh** (*State of Health*): Lo stato di salute stimato della batteria, calcolato come rapporto rispetto alla capacità nominale iniziale.
- **rul** (*Remaining Useful Life*): Il numero residuo di cicli utili prima del raggiungimento della fine della vita utile (*End of Life*).

Nel processo di pre-processing dei dati (implementato nel modulo `src/data.py`), vengono considerate solo le batterie che presentano almeno 30 cicli registrati, ottenendo così un dataset filtrato finale composto da 19 batterie e 1322 campioni complessivi. Per agevolare la convergenza e la stabilità numerica degli algoritmi di ottimizzazione, i cicli operativi vengono normalizzati nell'intervallo $[0, 1]$ dividendo l'indice progressivo per il massimo valore di cicli registrato (168).

1.3 Il Problema della Regressione Non Lineare

Dal punto di vista dell'Analisi Numerica, la previsione del degrado si traduce in un classico problema di *curve fitting*. L'obiettivo è individuare una funzione matematica $F(x; \theta)$, dipendente da un vettore di parametri incogniti θ , che approssimi al meglio un set di m misurazioni sperimentali (x_i, y_i) .

Per misurare la qualità dell'approssimazione, si definisce il vettore residuo r , in cui l' i -esimo elemento è dato dalla discrepanza tra il valore reale e quello predetto:

$$r_i = y_i - F(x_i; \theta) \quad (1)$$

Secondo il Metodo dei Minimi Quadrati, il vettore dei parametri ottimale θ^* si ottiene minimizzando la norma euclidea al quadrato del residuo (la *Loss function*):

$$\mathcal{L}(\theta) = \|r\|_2^2 = \sum_{i=1}^m (y_i - F(x_i; \theta))^2 \quad (2)$$

Mentre per i sistemi lineari è possibile ricavare la soluzione esatta risolvendo le Equazioni Normali ($X^T X \theta = X^T y$), per i modelli non lineari — come quello che governa il decadimento chimico — l'approccio analitico diretto non è percorribile. È pertanto necessario ricorrere a metodi iterativi di discesa per navigare lo spazio dei parametri e individuare il minimo globale della funzione $\mathcal{L}(\theta)$.

1.4 Obiettivo dello Studio

Lo scopo principale di questo progetto non risiede nel semplice addestramento di un modello black-box, bensì nell'analisi numerica comparativa di quattro differenti metodi di ottimizzazione continua: il Gradient Descent classico, l'algoritmo Quasi-Newton L-BFGS-B, e i moderni ottimizzatori stocastici derivati dal Deep Learning, ovvero Adam e Lion. Questi metodi verranno impiegati per fittare la seguente equazione parametrica non lineare:

$$y = \alpha e^{-\beta x} + \gamma \quad (3)$$

Lo studio esplorerà le dinamiche di convergenza, la gestione dei vincoli fisici e l'efficienza computazionale di ciascun algoritmo, dimostrando il legame profondo tra la matematica numerica tradizionale e l'Intelligenza Artificiale moderna.

2 Modello Fisico e Vincoli

2.1 L'Equazione Esponenziale

Il degrado della capacità di una batteria agli ioni di litio nel tempo (misurato in cicli di carica/scarica) non segue un andamento lineare, bensì è governato da dinamiche chimiche complesse che si manifestano con un decadimento iniziale rapido seguito da una stabilizzazione asintotica. Per catturare questo comportamento fenomenologico, si adotta il seguente modello fisico esponenziale a tre parametri:

$$y(x) = \alpha e^{-\beta x} + \gamma \quad (4)$$

dove x rappresenta il numero di cicli operativi e $y(x)$ la capacità residua della batteria. I parametri del modello assumono un significato fisico preciso:

- α (**Fattore di decadimento iniziale**): Rappresenta la quota di capacità che la batteria perde durante la prima fase di usura rapida.

- β (**Tasso di degrado**): Determina la velocità con cui avviene il decadimento esponenziale; valori maggiori indicano una perdita di prestazioni più repentina.
- γ (**Capacità asintotica residua**): Indica il valore limite teorico verso cui tende la capacità della batteria dopo un numero infinito di cicli, ovvero lo stato di salute di fondo non intaccato dall'usura rapida iniziale.

2.2 Spazio di Ottimizzazione e Vincoli

Affinché il modello generi previsioni fisicamente sensate e interpretabili, lo spazio di ottimizzazione non può essere illimitato. Infatti, una batteria non può avere una capacità negativa, né può esibire una crescita esponenziale della stessa nel tempo o avere un tasso di degrado negativo. È pertanto indispensabile imporre dei limiti fisici stringenti, traducibili in *Box Constraints* strettamente positivi per tutti e tre i parametri:

$$\alpha > 0, \quad \beta > 0, \quad \gamma > 0 \quad (5)$$

La gestione di tali vincoli varia significativamente a seconda dell'algoritmo di ottimizzazione impiegato. Metodi avanzati come L-BFGS-B li supportano nativamente sfruttando l'approccio matematico dell'*Active Set*: qualora un parametro urti il limite imposto, la direzione di ricerca viene bloccata lungo quella specifica dimensione finché il gradiente non indichi nuovamente una regione ammissibile interna. Tuttavia, algoritmi nati per scenari non vincolati (come Gradient Descent, Adam o Lion) richiedono l'introduzione di una *Barrier Penalty* (o funzione di penalità) all'interno della Loss function. Qualora i parametri violino i vincoli di positività imposti, viene sommato un costo artificiale molto elevato all'errore, forzando matematicamente l'ottimizzatore a deviare la propria traiettoria e a rientrare nella regione ammissibile dello spazio di ottimizzazione.

3 Rassegna degli Algoritmi di Ottimizzazione Continua

Il problema di identificare i parametri ottimali del modello fenomenologico delineato richiede la minimizzazione della Loss function, una procedura che per modelli non lineari si affida a metodi iterativi di discesa. Di seguito vengono analizzati i quattro approcci numerici implementati in questo studio.

3.1 Metodi del Primo Ordine: Gradient Descent (GD)

Il Gradient Descent rappresenta il fondamento dell'ottimizzazione continua. Esso aggiorna iterativamente i parametri muovendosi nella direzione di massima decrescita locale, dettata dal gradiente negativo della funzione di costo:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(k)}) \quad (6)$$

dove η è il passo di discesa, noto anche come learning rate. Sebbene teoricamente solido per funzioni strettamente convesse, il GD soffre di forti rallentamenti, causati dal fenomeno dello *zig-zagging*, in presenza di valli strette e mal condizionamento dell'Hessiana. L'impiego del campionamento stocastico, tipico della variante SGD, mitiga tale limite introducendo un rumore che aiuta a sfuggire a minimi locali, pur senza risolvere del tutto le difficoltà legate alla curvatura dello spazio.

3.2 Metodi Adattivi e del Momento: Adam

L'algoritmo Adam, ovvero *Adaptive Moment Estimation*, supera i limiti del GD combinando due concetti chiave dell'Analisi Numerica: l'inerzia e il passo adattivo. Adam tiene traccia di una media mobile esponenziale sia dei gradienti passati, definiti come Primo Momento per stabilizzare la traiettoria, sia dei gradienti al quadrato, ovvero il Secondo Momento necessario a stimare la varianza. Questo meccanismo permette di smorzare le oscillazioni tipiche dello zig-zagging e di calibrare automaticamente un learning rate specifico per ogni parametro, rendendolo l'algoritmo dominante nel moderno panorama del Machine Learning.

3.3 Metodi Quasi-Newton: L-BFGS-B

Mentre GD e Adam si basano unicamente sulle derivate prime, i metodi Quasi-Newton cercano di approssimare l'Hessiana della funzione di costo per catturarne l'esatta curvatura geometrica tramite derivate seconde, ma senza il costo proibitivo di calcolarla e invertirla esplicitamente come nel Metodo di Newton standard. L'algoritmo L-BFGS-B aggiorna iterativamente una matrice di approssimazione utilizzando solo i gradienti e le posizioni degli ultimi step, seguendo un approccio a memoria limitata. A differenza dei precedenti, questo algoritmo possiede una profonda percezione geometrica della superficie di Loss e, soprattutto, integra nativamente la gestione formale dei limiti sui parametri, noti come Box constraints, tramite logiche di *Active Set*, rivelandosi teoricamente perfetto per spazi di ottimizzazione a bassa dimensionalità ben posti.

3.4 Innovazioni Estreme: Lion

Lion, acronimo di *EvoLved Sign Momentum*, rappresenta un'estremizzazione recente sviluppata per addestrare modelli massivi. L'algoritmo conserva l'uso dell'inerzia, ma elimina completamente il calcolo della magnitudine del gradiente, sfruttandone esclusivamente l'informazione binaria del segno positivo o negativo. L'aggiornamento dei parametri avviene a falcate di lunghezza fissa pari ad η :

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \cdot \text{sign}(\mathbf{m}^{(k)}) \quad (7)$$

Se da un lato questo approccio dimezza i requisiti di memoria poiché non deve tracciare la varianza, dall'altro la mancanza di modulazione dell'intensità del passo penalizza fortemente l'algoritmo nelle fasi finali della convergenza, rischiando di innescare rimbalzi perpetui attorno all'ottimo matematico, specialmente su scale ridotte.

4 Architettura Software e Metodologia

4.1 Struttura del Codice

L'implementazione software è stata sviluppata sfruttando il framework PyTorch, scelto per la sua eccellente capacità di calcolare gradienti in modo automatico su tensori multidimensionali (Autograd). Il codice sorgente è organizzato seguendo un paradigma altamente modulare, suddiviso in due aree principali:

- Il package `src/`, che funge da libreria core. Al suo interno sono incapsulate le definizioni matematiche del modello esponenziale di degrado, la gestione rigorosa dei vincoli fisici e le classi specializzate per ciascun ottimizzatore (Adam, L-BFGS-B, Lion e GD).
- La directory `scripts/`, dedicata all'esecuzione operativa. Qui risiedono gli script di alto livello preposti al caricamento del dataset, all'avvio parallelo delle sessioni di training e alla generazione dei grafici per l'analisi comparativa dei risultati.

Tale separazione garantisce un'ottima scalabilità e manutenibilità del progetto, permettendo di testare nuovi algoritmi di ottimizzazione senza alterare la logica di business fondamentale o la pipeline di caricamento dati.

4.2 Pipeline Sperimentale

L'esperimento è stato condotto seguendo una rigorosa pipeline metodologica. Inizialmente, i dati grezzi estratti dal dataset NASA sono stati importati e preprocessati per isolare la curva di degrado della capacità in funzione dei cicli operativi. Successivamente, la funzione di costo basata sull'Errore Quadratico Medio (MSE) è stata calcolata iterativamente.

Per ogni ottimizzatore testato, sono state registrate due metriche fondamentali:

1. **Efficienza temporale:** Il tempo effettivo impiegato per raggiungere la convergenza, misurato in secondi, al fine di valutare il costo computazionale dell'algoritmo.
2. **Dinamica di convergenza:** La tracciatura passo passo della Loss per analizzare il comportamento geometrico della discesa, ovvero la capacità dell'algoritmo di evitare zig-zagging e di gestire correttamente i vincoli fisici imposti al modello.

Questa duplice valutazione ha permesso di confrontare oggettivamente le prestazioni dei vari metodi numerici, mettendo in luce i compromessi intrinseci tra velocità di esecuzione e stabilità teorica.

5 Analisi dei Risultati

5.1 Confronto delle Metriche Computazionali

L'analisi delle performance ha evidenziato divari sostanziali tra gli ottimizzatori testati, ben visibili nel grafico a barre riassuntivo (Figura 1). L-BFGS-B ha dimostrato una supremazia assoluta in termini di efficienza temporale, raggiungendo la convergenza esatta in circa 0.01 secondi. Al contrario, gli ottimizzatori stocastici di primo ordine derivati dal Deep Learning, quali Adam e Lion, hanno richiesto svariate decine di secondi per approssimarsi al medesimo livello di Loss finale.

Tale enorme discrepanza computazionale trova spiegazione nella natura dello spazio dei parametri. Essendo il problema fortemente vincolato e confinato in sole tre dimensioni (α, β, γ) , la matrice Hessiana è di dimensione 3×3 . Di conseguenza, la stima quasi-esatta della curvatura operata da L-BFGS-B richiede un costo di memoria e di calcolo irrisorio. L'algoritmo può così sfruttare l'informazione del secondo ordine per convergere al minimo in pochissime iterazioni analitiche, surclassando la strategia stocastica di Adam che, pur essendo intrinsecamente più robusta in scenari altamente rumorosi, impiega un numero immensamente maggiore di step esplorativi sui mini-batch per ricostruire solo indirettamente la topologia della valle.

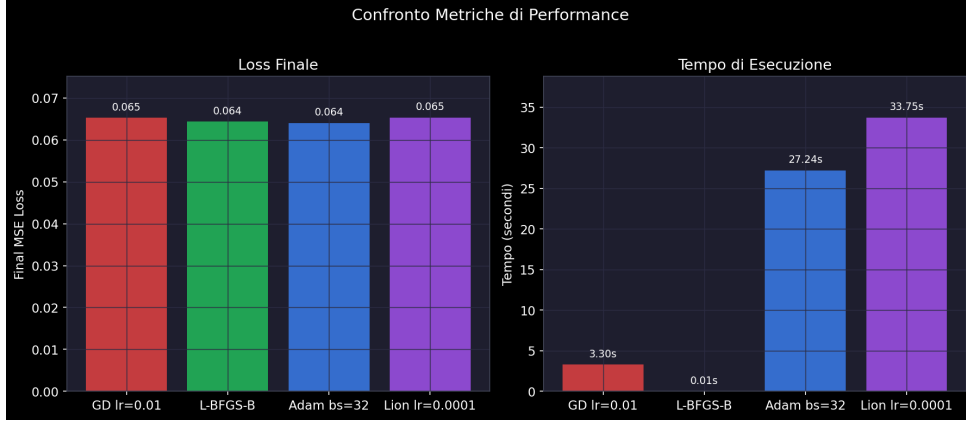


Figura 1: Confronto dei tempi di esecuzione e Loss finale tra i diversi algoritmi.

5.2 Dinamica di Convergenza Geometrica e Traiettorie

La tracciatura tridimensionale della traiettoria di minimizzazione (Figura 2) permette di apprezzare visivamente il comportamento geometrico dei metodi analizzati sulla superficie di Loss.

Mentre L-BFGS-B esegue una discesa diretta e deterministica verso il bacino di minimo — planando con estrema precisione e bloccando i movimenti non validi grazie alla gestione esatta dell'Active Set — Adam manifesta un percorso più esplorativo e ridondante. Sebbene l'inerzia (Momento) e l'adattamento del passo aiutino Adam a mitigare lo zig-zagging rispetto al Gradient Descent puro, l'assenza di derivate seconde reali lo costringe a muoversi per approssimazioni successive, allungando sensibilmente la traiettoria geometrica prima di calarsi nel minimo globale.

Ancora più istruttivo, dal punto di vista dell'Analisi Numerica, è il collasso geometrico di Lion. Poiché Lion si basa esclusivamente sul segno del gradiente scartandone completamente la magnitudine, esso compie aggiornamenti a falcate di ampiezza fissa pari al learning rate η . Sulla superficie di Loss di questo specifico problema, caratterizzata da un imbuto asimmetrico e piuttosto ripido, Lion non è matematicamente in grado di ridurre dolcemente l'intensità del passo man mano che si avvicina all'ottimo: questo limite strutturale lo porta a innescare micro-rimbaldi perpetui (oscillazioni continue ad alta frequenza) attorno al minimo esatto, confermando le sue forti criticità teoriche nell'ottimizzazione su bassa dimensionalità e scale ridotte.

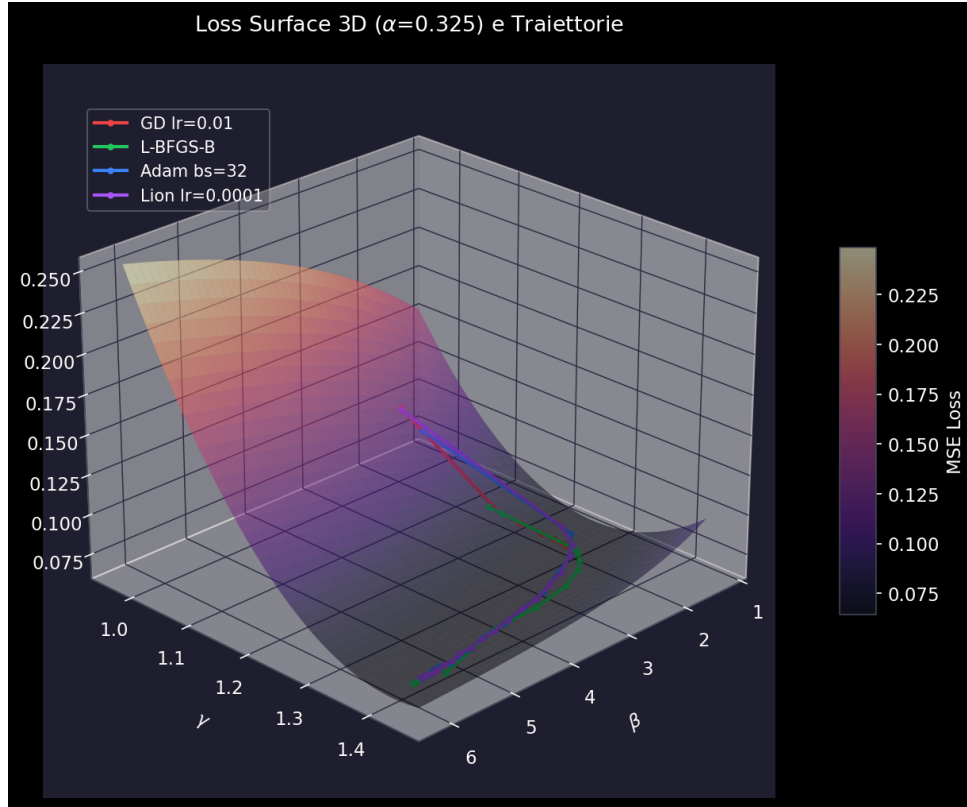


Figura 2: Superficie di Loss 3D con le traiettorie di discesa degli ottimizzatori nello spazio dei parametri.

6 Modello Fisico vs Deep Learning

6.1 Implementazione della Baseline Neurale (MLP)

Per contestualizzare ulteriormente le prestazioni del modello fisico, è stata introdotta un'architettura puramente *data-driven* basata sul Deep Learning: un Multi-Layer Perceptron (MLP). Nello specifico, la rete è strutturata in tre layer densi (*fully-connected*) intervallati da funzioni di attivazione non lineari ReLU (*Rectified Linear Unit*):

- **Layer di ingresso:** mappa 1 feature (i cicli operativi) in 32 neuroni nascosti.
- **Hidden layer centrale:** riceve in input i 32 neuroni e ne restituisce in output altrettanti.
- **Layer di uscita:** contrae i 32 neuroni nella singola predizione continua finale (la capacità residua).

Tale architettura comporta l'aggiornamento di un totale esatto di 1153 parametri allenabili (distribuiti tra pesi sinaptici e bias). La rete è stata addestrata per apprendere la mappatura input-output dai dati grezzi, eludendo totalmente la conoscenza pregressa dell'equazione esponenziale teorica.

6.2 Confronto Teorico-Pratico e Sovra-parametrizzazione

I risultati dell'addestramento della rete neurale evidenziano i limiti strutturali degli approcci puramente *data-driven*. Su un problema di così bassa dimensionalità e ben caratterizzato

fisicamente, il Deep Learning classico si rivela fortemente sub-ottimale rispetto all'Analisi Numerica tradizionale (L-BFGS-B).

In primo luogo, l'MLP soffre della maledizione della sovra-parametrizzazione: disponendo di 1153 parametri per approssimare una curva governata intrinsecamente da sole 3 variabili reali, la rete tende fisiologicamente a "imparare il rumore" presente nei dati sperimentali (leggero overfitting), producendo traiettorie meno lisce e asintoticamente meno robuste rispetto all'esponenziale fisico. In secondo luogo, si assiste all'annientamento totale dell'interpretabilità: mentre α , β e γ offrono una chiara misurazione chimico-fisica dello stato della batteria (tasso di degrado, asintoto, ecc.), i pesi dell'MLP rappresentano una scatola nera impenetrabile (*black-box*). Infine, dal punto di vista computazionale, addestrare l'MLP ha richiesto un incremento spropositato dei tempi di calcolo (nell'ordine del 30000% in più) rispetto alla frazione di secondo impiegata da L-BFGS-B, senza peraltro apportare alcun reale beneficio predittivo a lungo termine.

Ciò dimostra che, in scenari ingegneristici dove le equazioni governanti sono note o approssimabili, un modello fenomenologico fisicamente vincolato accoppiato a metodi Quasi-Newton rimane la scelta metodologica superiore rispetto al Deep Learning purista.

7 Conclusioni

Il presente studio ha affrontato il problema della previsione del degrado delle batterie agli ioni di litio confrontando diverse metodologie di ottimizzazione continua applicate a un modello esponenziale fenomenologico. I risultati empirici hanno dimostrato un perfetto allineamento con i dettami teorici dell'Analisi Numerica.

È emerso chiaramente come, su scenari a bassa dimensionalità e ben posti fisicamente, gli approcci classici basati sull'approssimazione dell'Hessiana (come l'algoritmo Quasi-Newton L-BFGS-B) offrano prestazioni nettamente superiori rispetto alle varianti stocastiche del primo ordine dominanti nel Machine Learning odierno (Adam, Lion). La percezione geometrica della curvatura dello spazio e la gestione rigorosa dei vincoli tramite Active Set hanno permesso a L-BFGS-B di raggiungere la convergenza esatta in frazioni di secondo, surclassando metodi che richiedono decine di secondi di esplorazione stocastica o che, come nel caso di Lion, palesano limiti strutturali sul controllo del passo.

Infine, il confronto diretto con una baseline Deep Learning (MLP) ha ribadito le inefficienze intrinseche degli approcci *black-box* su sistemi ben definiti: l'impiego di reti neurali massivamente parametrizzate su problemi descrivibili da poche equazioni fondamentali si traduce unicamente in un ingiustificabile incremento dei costi computazionali, nella perdita totale di interpretabilità e in una maggiore vulnerabilità al rumore (overfitting). Si conclude pertanto che, ovunque sia disponibile un solido modello fisico di riferimento, la combinazione di quest'ultimo con algoritmi classici dell'Analisi Numerica rimane la strategia ingegneristica più robusta, efficiente e affidabile.